

Patterns in Parallel Programming

COMP 445

Relevant reading and reference:

[Chapter 1 of Intermediate PDC](#)

Parallel Patterns in Programs

... are industry-standard techniques and best-practices that have proven useful in many different parallel contexts.

... are built into popular platforms like MPI and OpenMP.

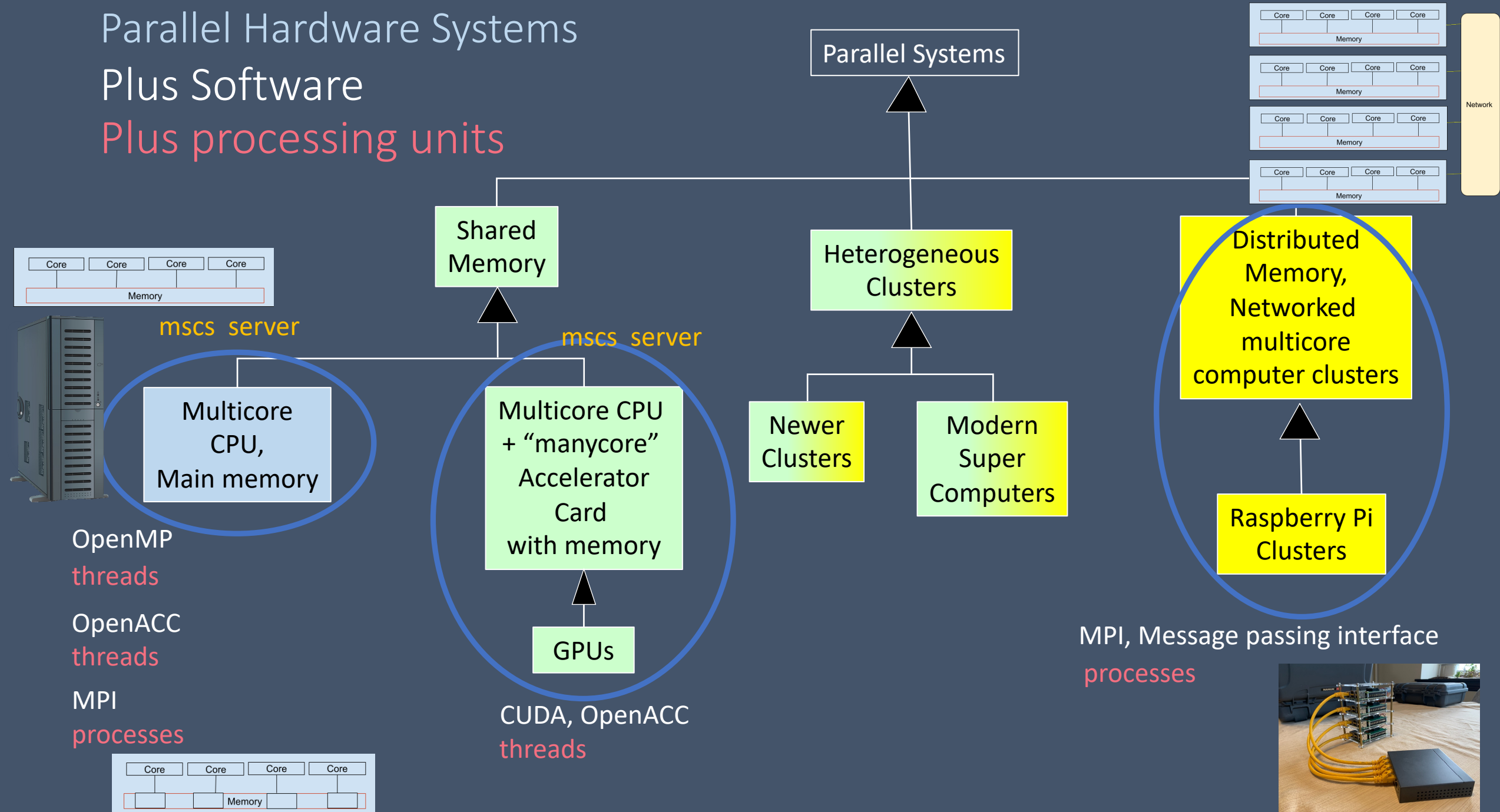
... are likely to be useful for the long-term, regardless of future PDC developments.

... provide a way to organize PDC concepts.

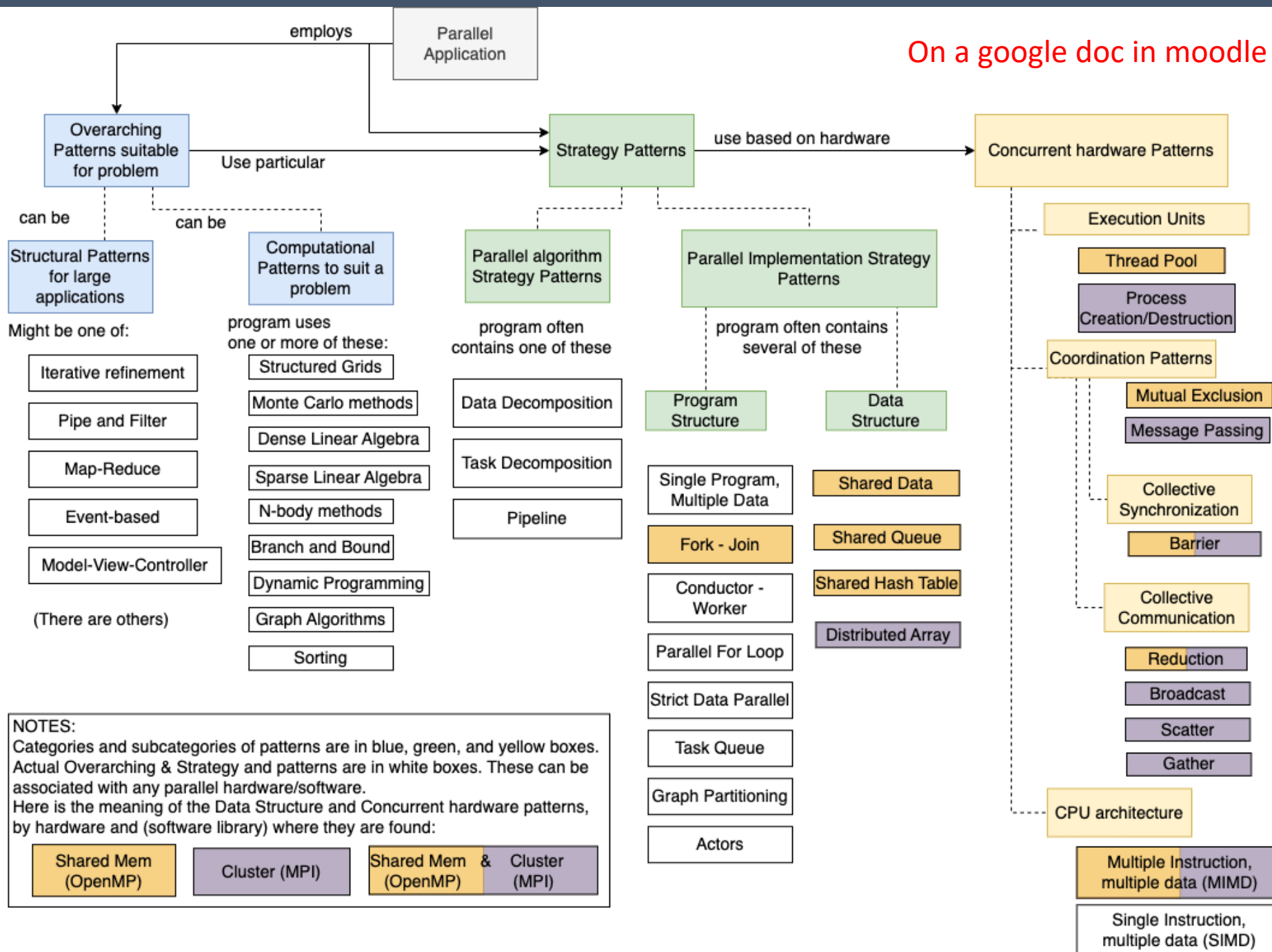
Patterns: tried and true methods in our code

Let's consider how to organize the types of software *patterns* that enable us to achieve parallelism

Parallel Hardware Systems Plus Software Plus processing units



On a google doc in moodle



Parallel Pattern Categories

These patterns can be categorized:

Computational Patterns:

overall approach to a problem
(originally sequential).

Algorithmic Strategies:

general approaches to devising parallel
algorithms of a computational pattern.

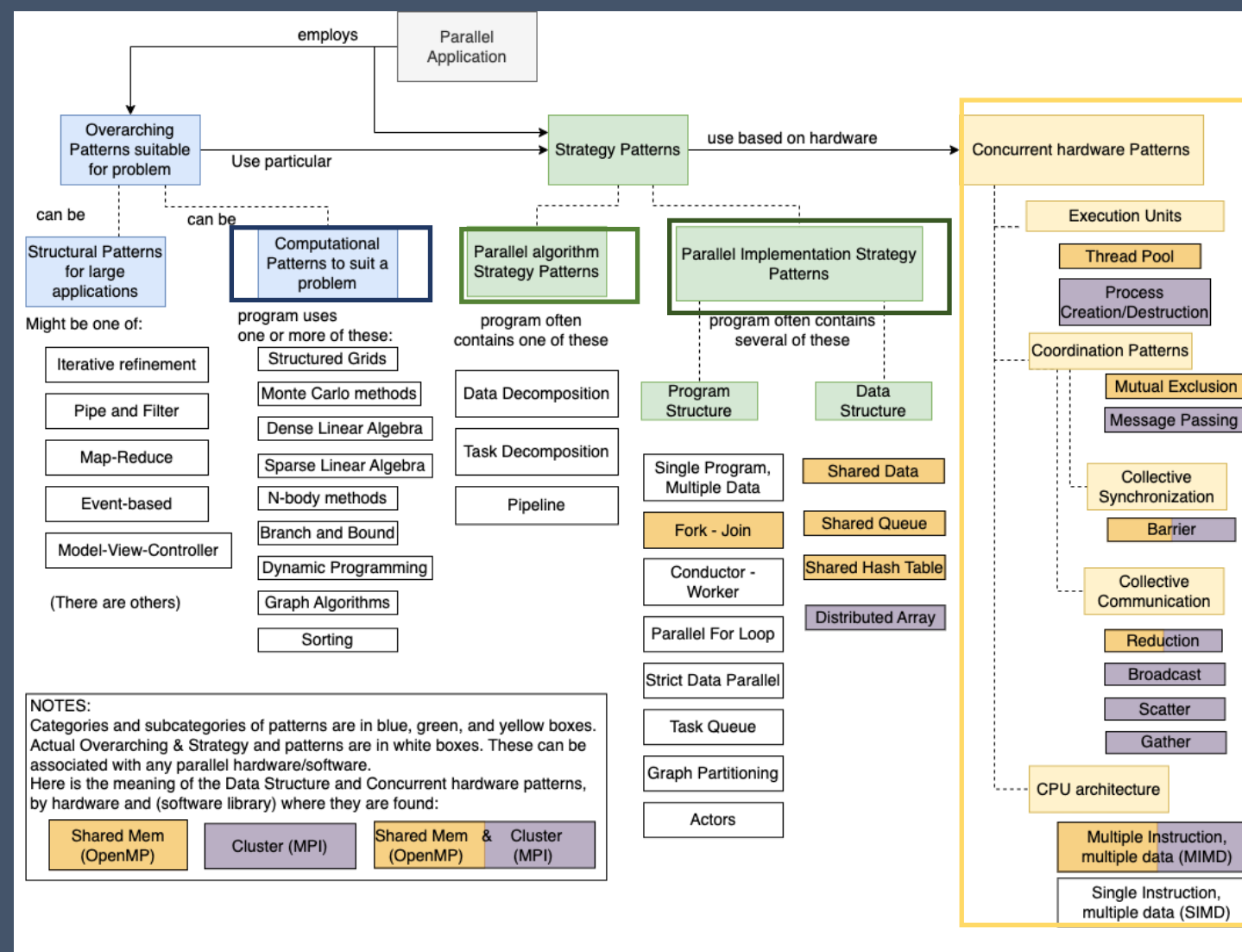
Implementation Strategies:

patterns used to implement a given
algorithmic strategy.

Concurrent Hardware (Communication & Synchronization):

patterns for
synchronizing/communicating between
the tasks in a given strategy.

--> These are usually found inside of the
libraries we use to build our code.



Parallel Algorithm Strategies

Most parallel programs use one of just three *parallel algorithm strategy patterns*:

Let's start with this one

Data decomposition: divide up the data and process it in parallel.

This approach is usually more scalable than the others

Task decomposition: divide the algorithm into functional tasks that are performed in parallel (to the extent possible).

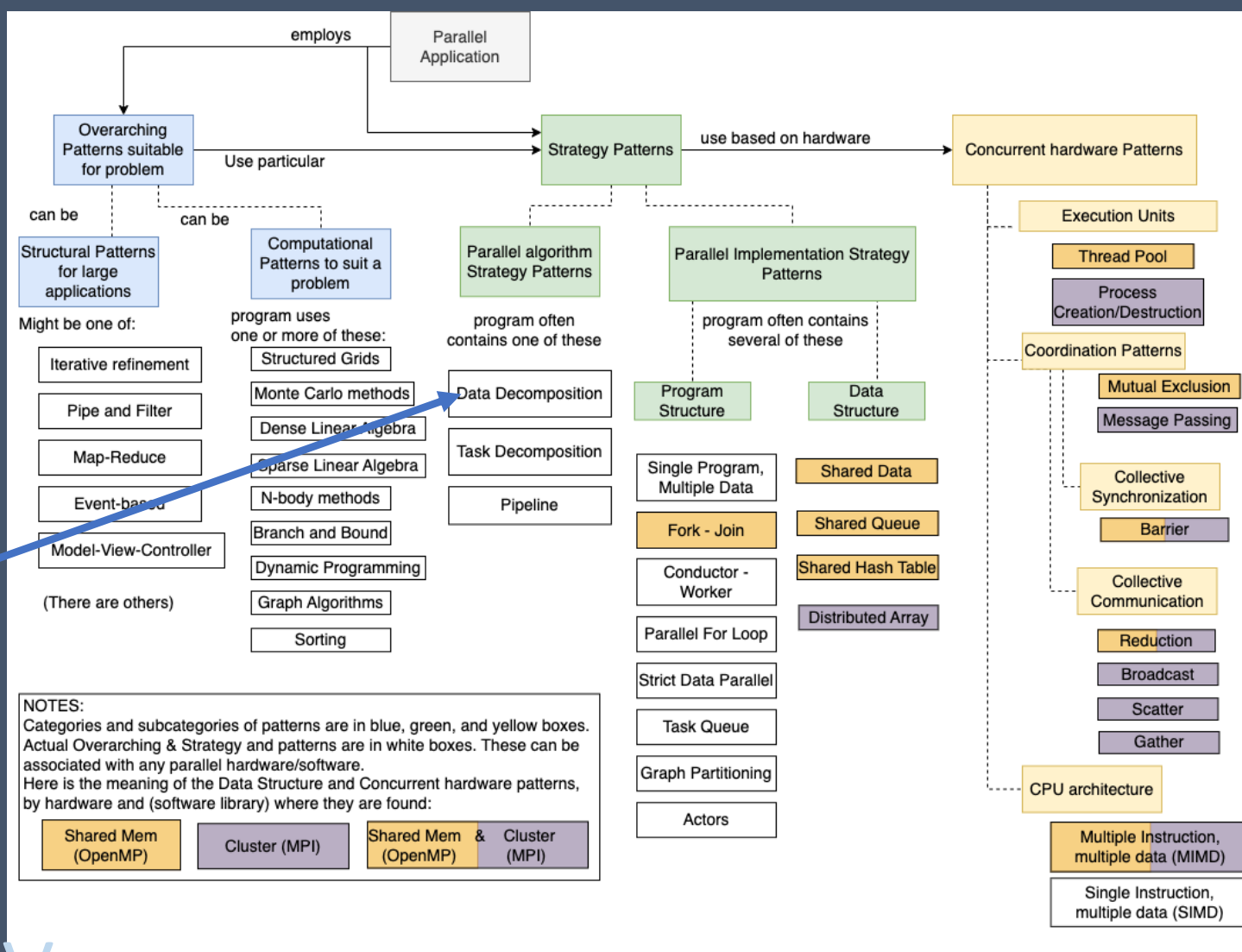
Also called
“functional decomposition”

Pipeline: divide the algorithm into linear stages, through which we “pump” the data.

Data Decomposition Algorithm Strategy

on shared memory machines with threads

Yet applies generally to other systems



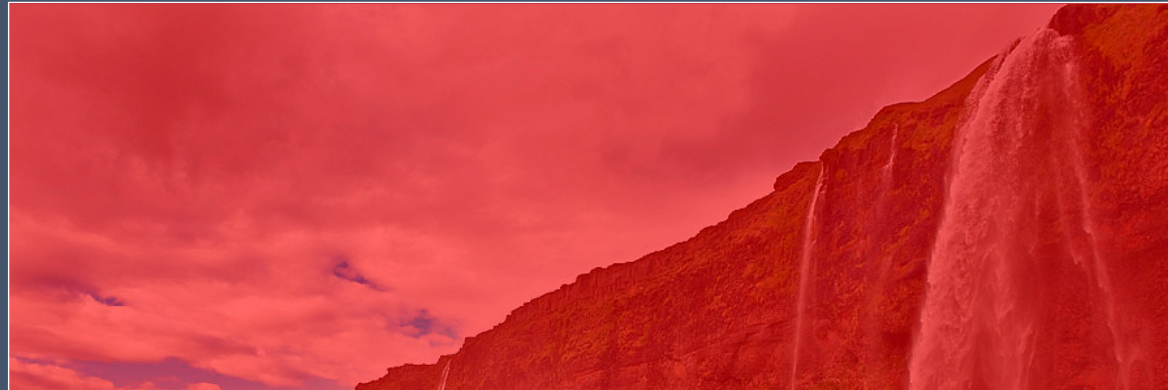
Data Decomposition: 1 Thread

Thread 0

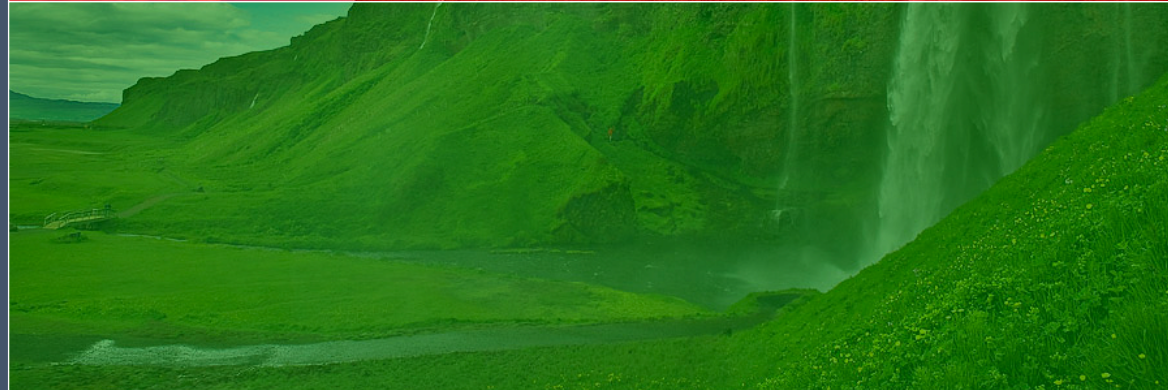


Data Decomposition: 2 Threads

Thread 0

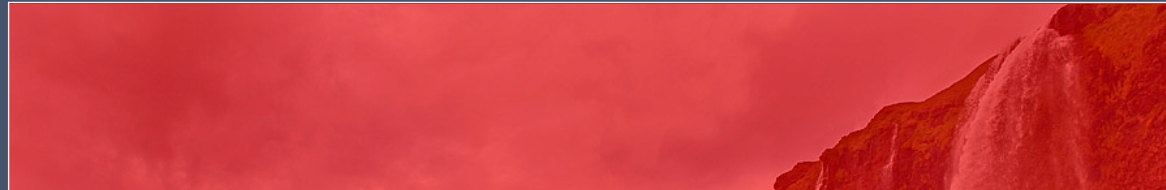


Thread 1



Data Decomposition: 4 Threads

Thread 0



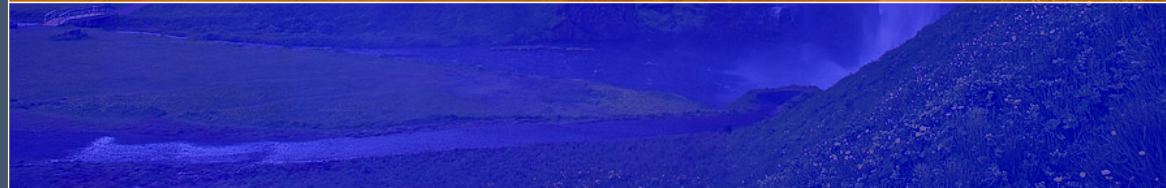
Thread 1



Thread 2



Thread 3



In OpenMP, this is accomplished by using “parallel for” in the pragma.
In this example, ‘chunks’ of the original data are worked on by each thread

Shared Memory Machines

False sharing is an issue!
Thread 0 could load part of thread 1 data into thread 0's cache

Thread 0

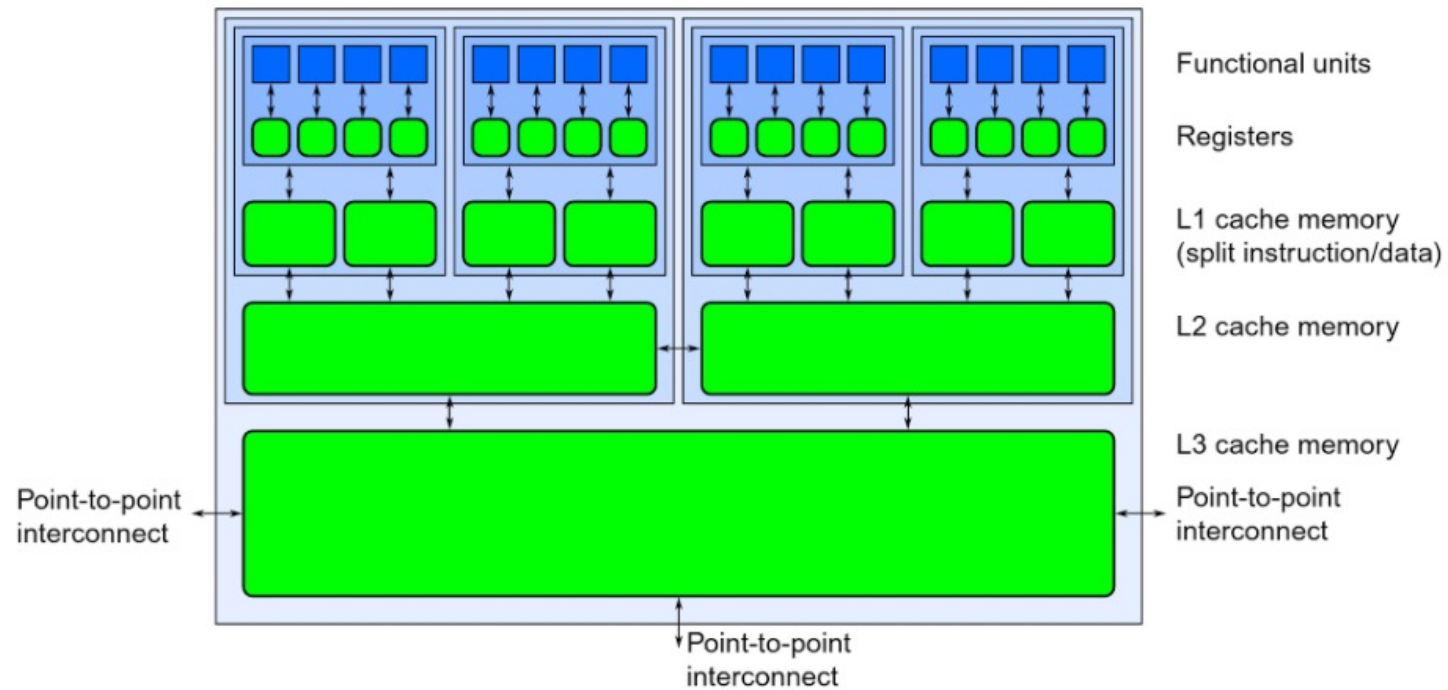
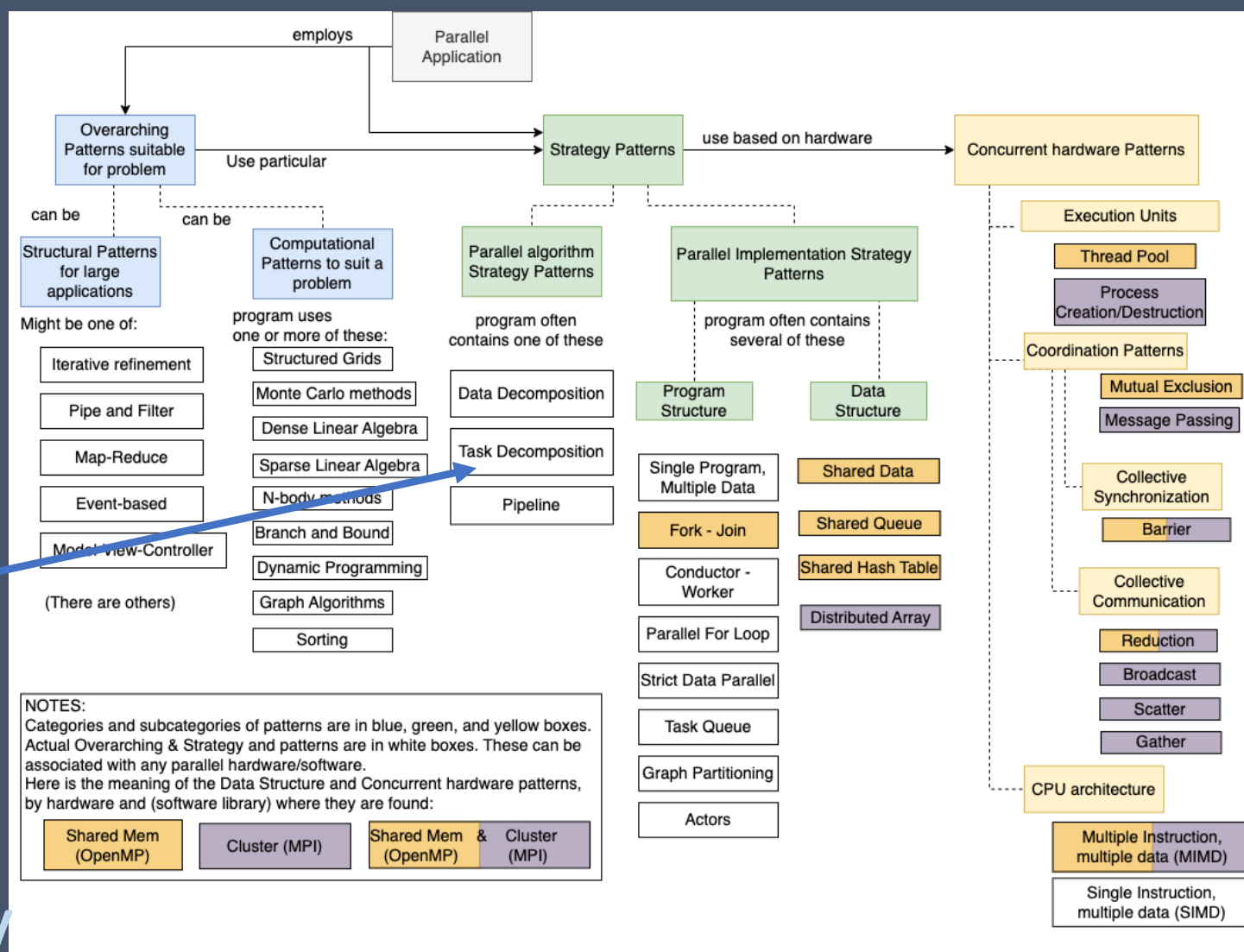


FIGURE 2.2

Multicore processor with hierarchical cache. Each core has multiple functional units and (typically) an instruction cache and a data cache. Larger, slower caches are then shared between increasing numbers of cores in a hierarchy.

Task Decomposition Algorithm Strategy

on shared memory machines with threads
Yet applies generally to other systems

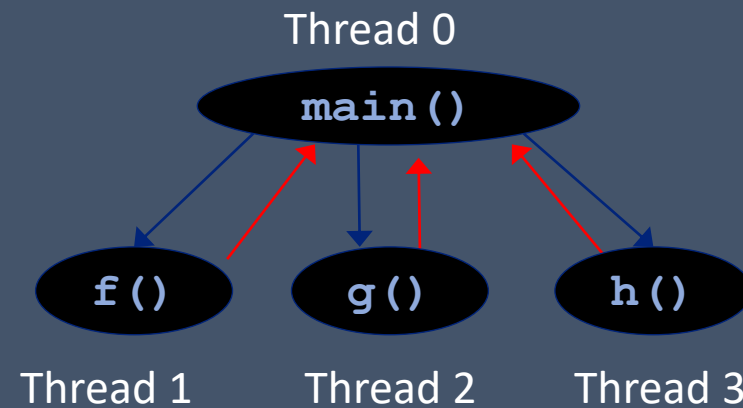


Algorithmic Strategy: Task Decomposition

Also called “functional decomposition”

The independent functions in a sequential computation can be “parallelized”:

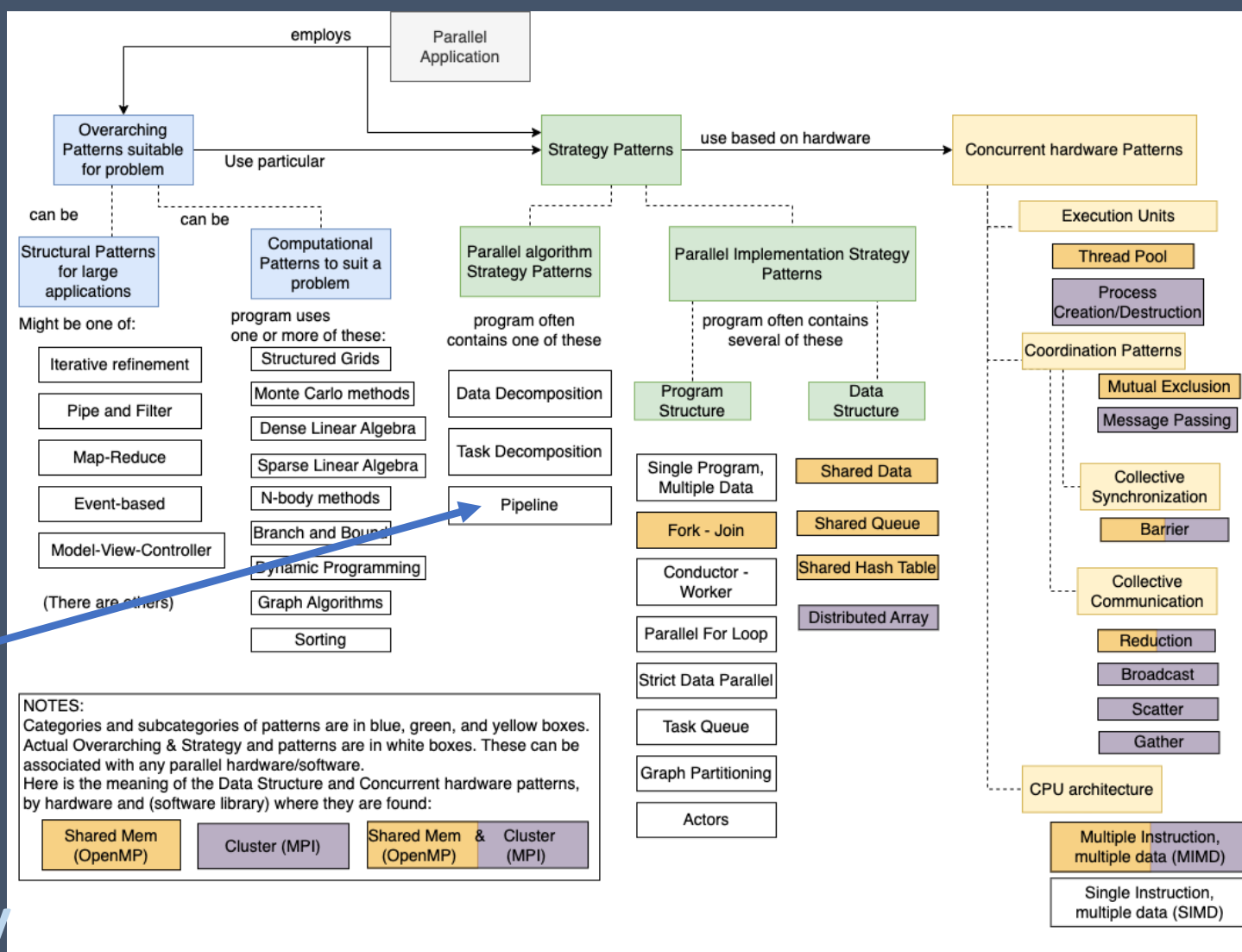
```
int main() {  
    x = f();  
    y = g();  
    z = h();  
    w = x + y + z;  
}
```



In OpenMP, this is accomplished by using ‘sections’.
See the 16.sections patternlet

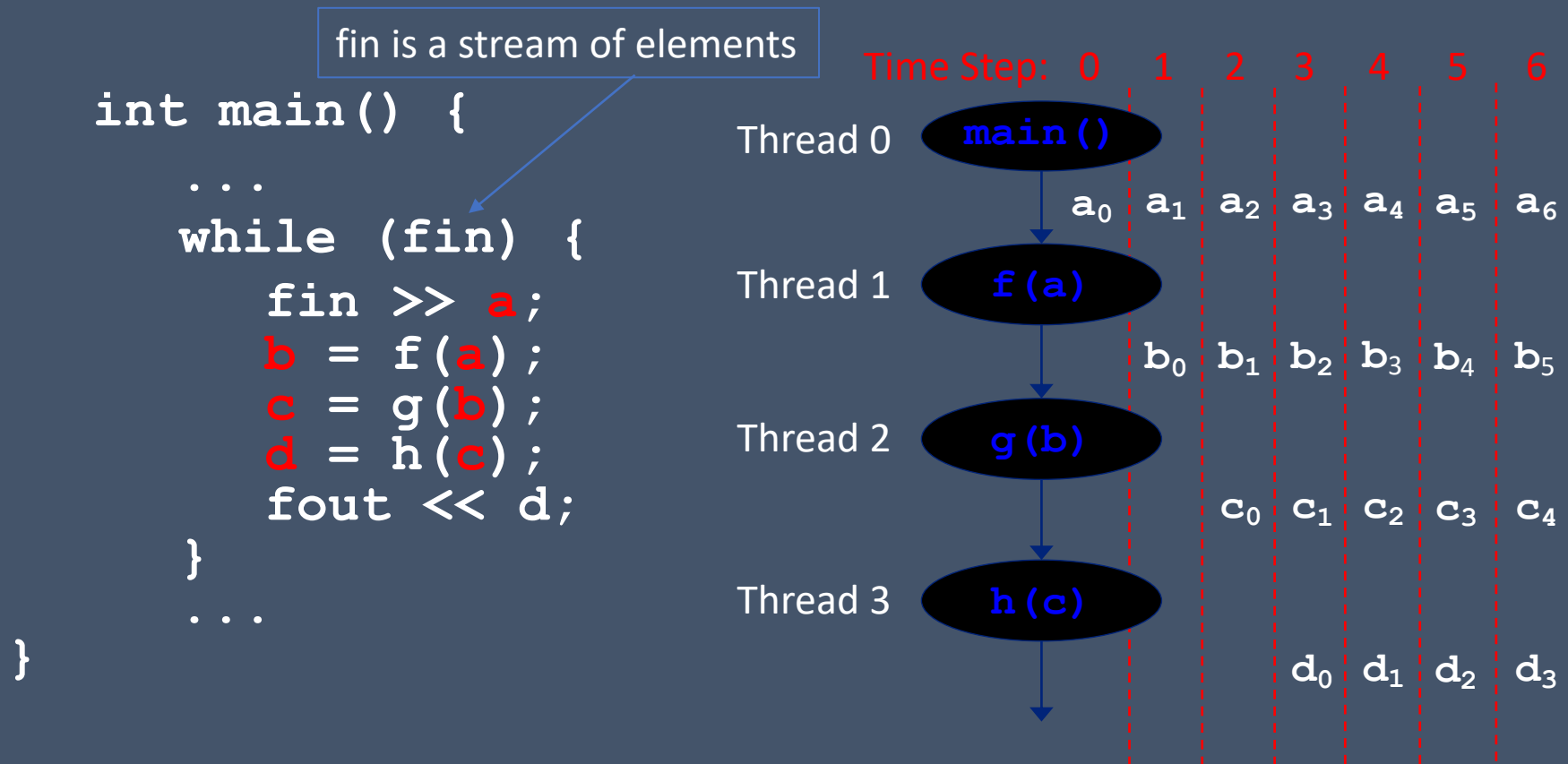
Pipeline Algorithm Strategy

on shared memory machines with threads
Yet applies generally to other systems

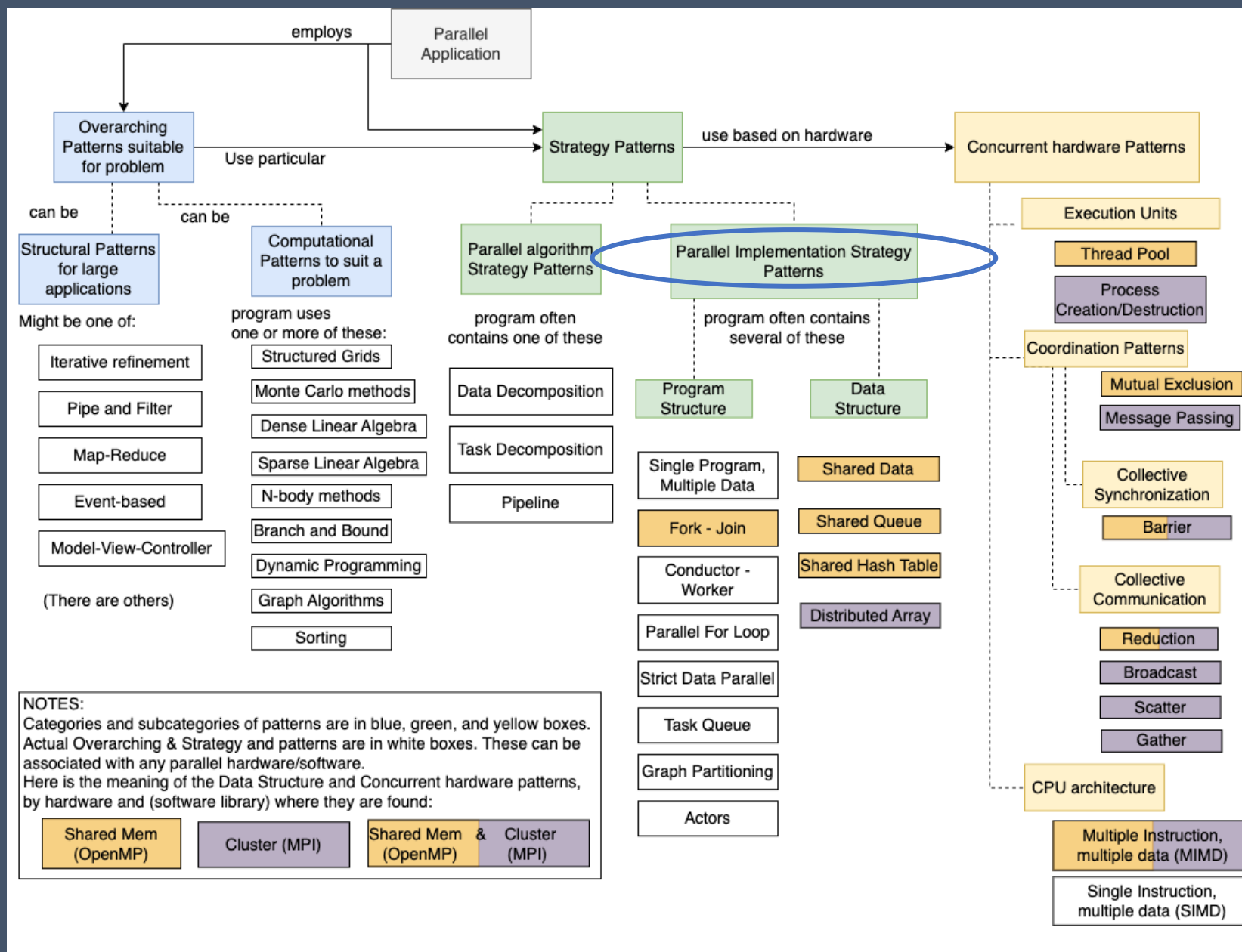


Algorithmic Strategy: Pipeline

- When functions are not independent:



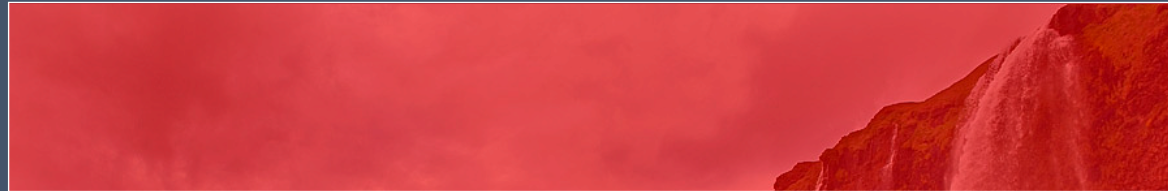
they can still be *pipelined*...



Implementation Strategies

How do we implement Data Decomposition?

Thread 0



Thread 1

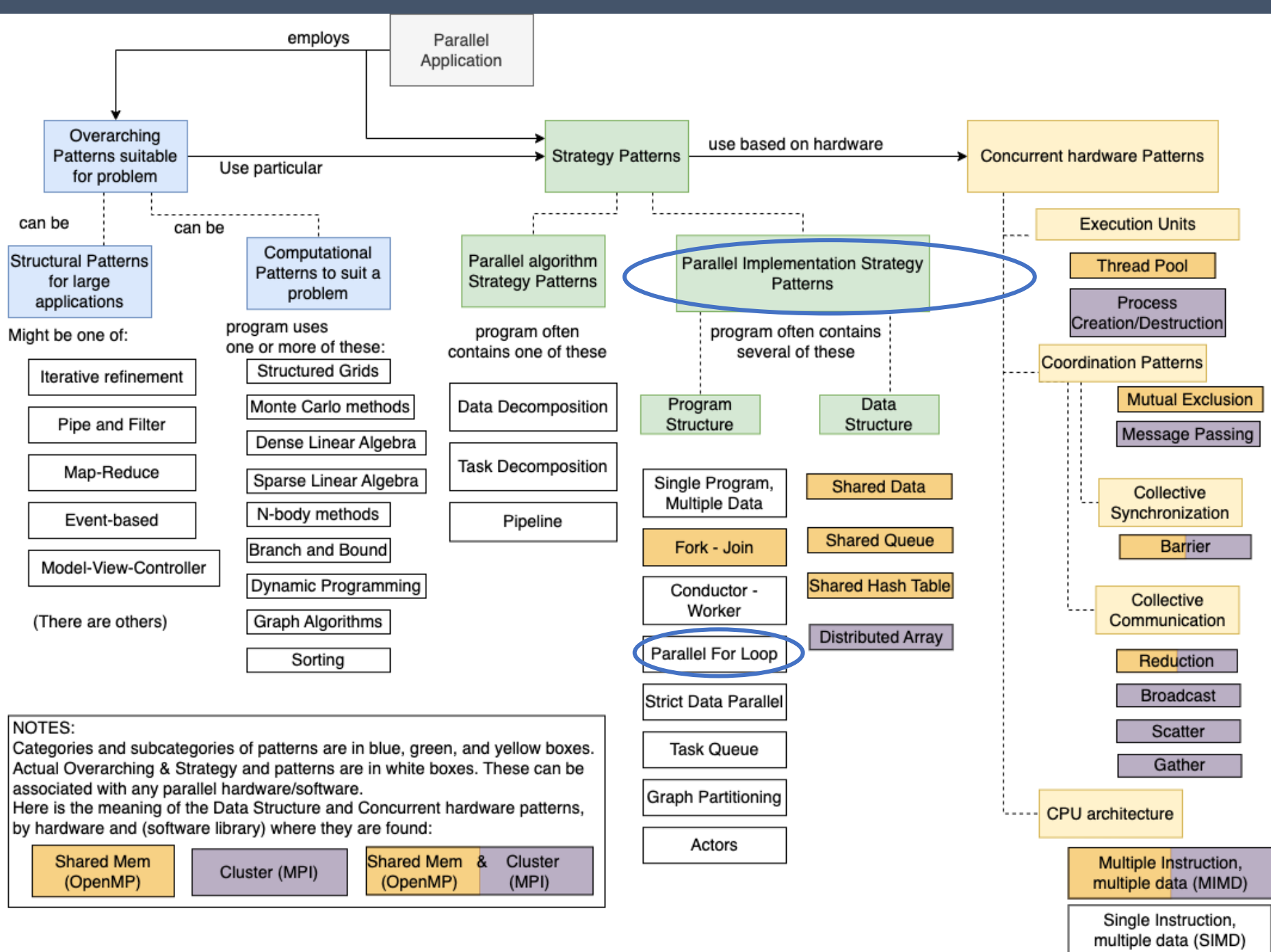


Thread 2



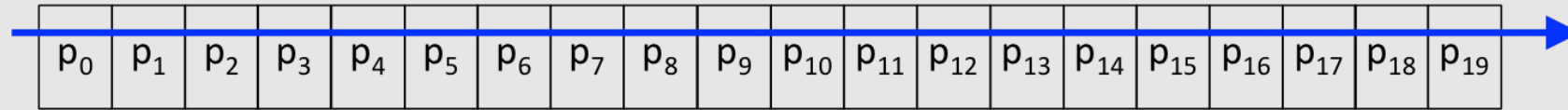
Thread 3



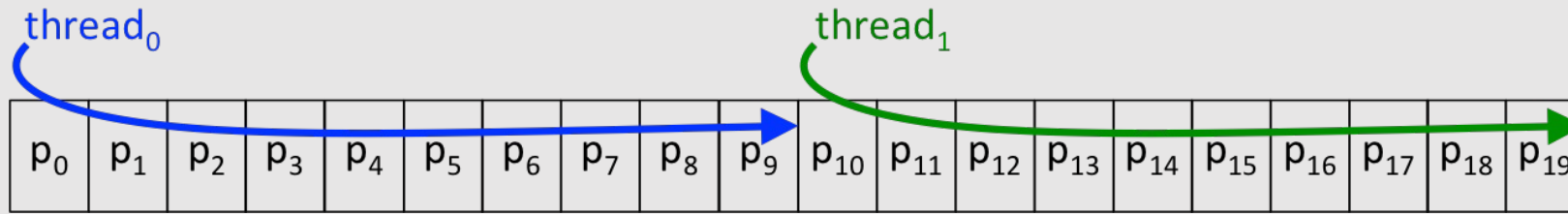


Implementation Strategy: Parallel Loop

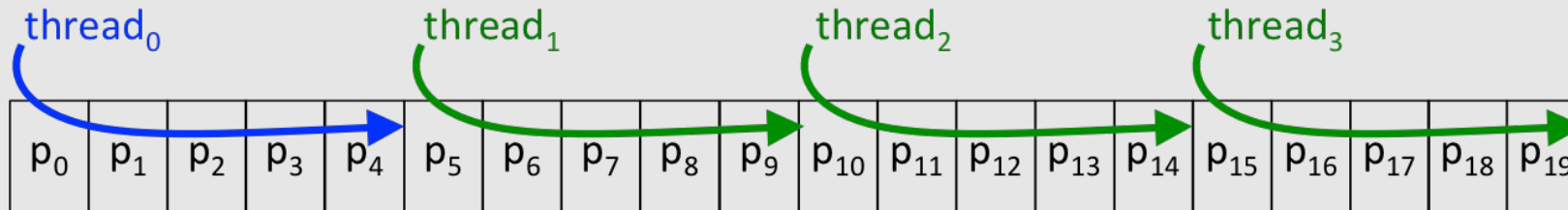
One Thread:



Two Threads:

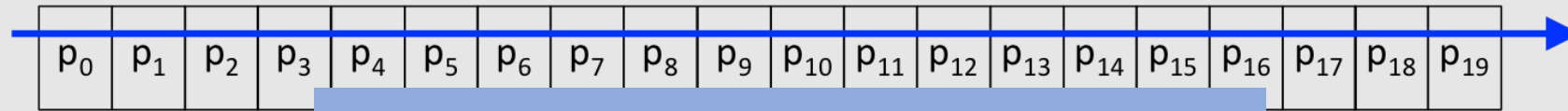


Four Threads:



Implementation Strategy: Parallel Loop

One Thread:



Two Threads

thread₀



Suppose the task was to add all the numbers in this array:
Reduce to a single sum

We will use another pattern for that!

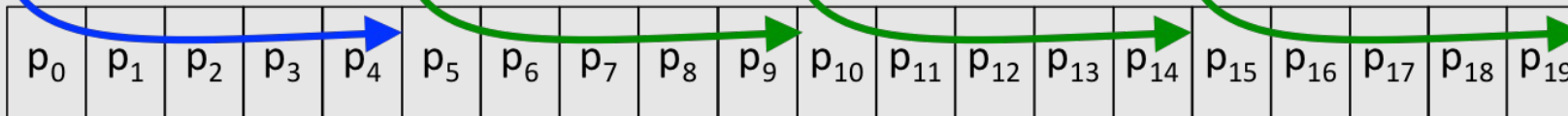
Four Threads:

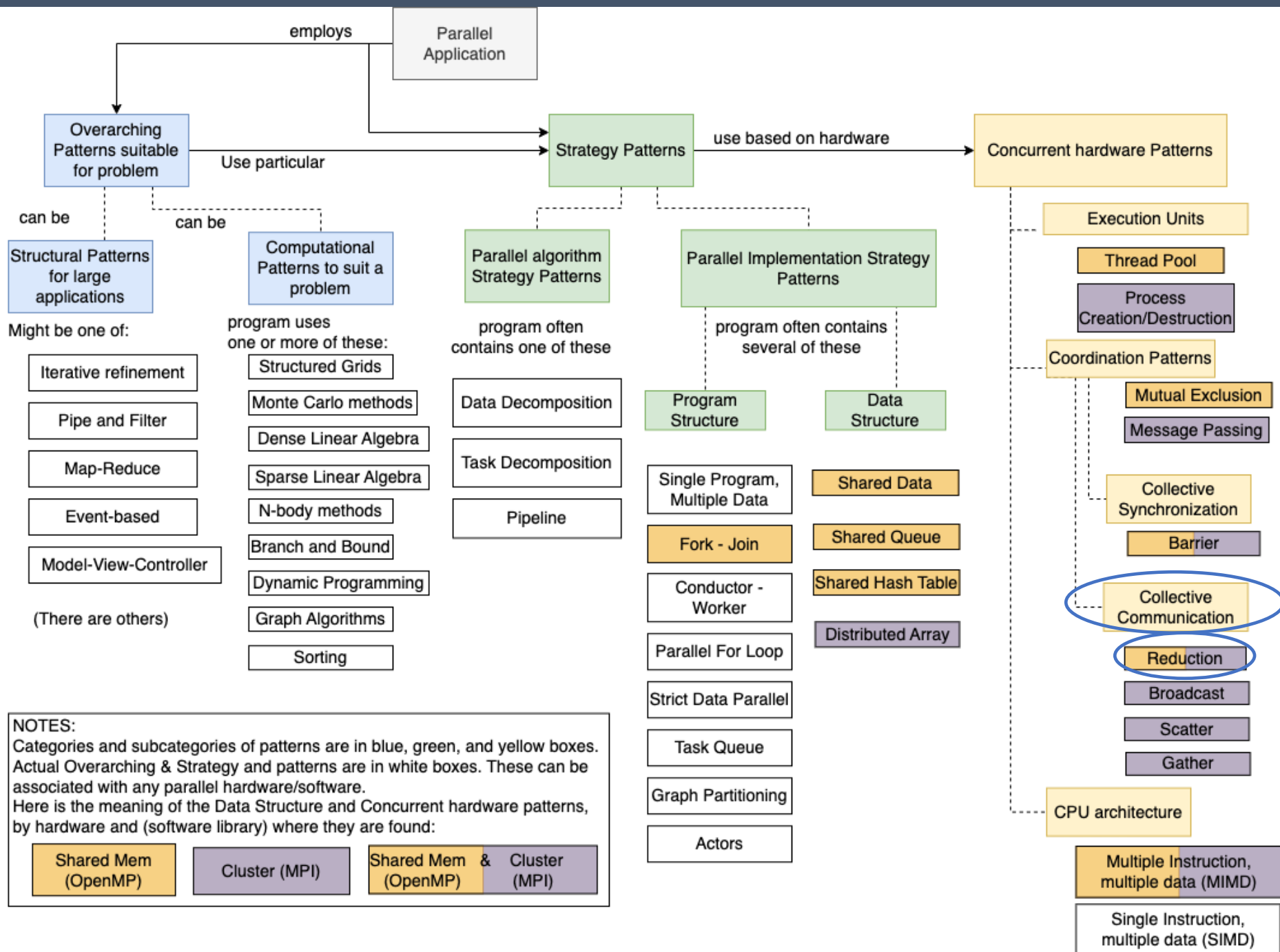
thread₀

thread₁

thread₂

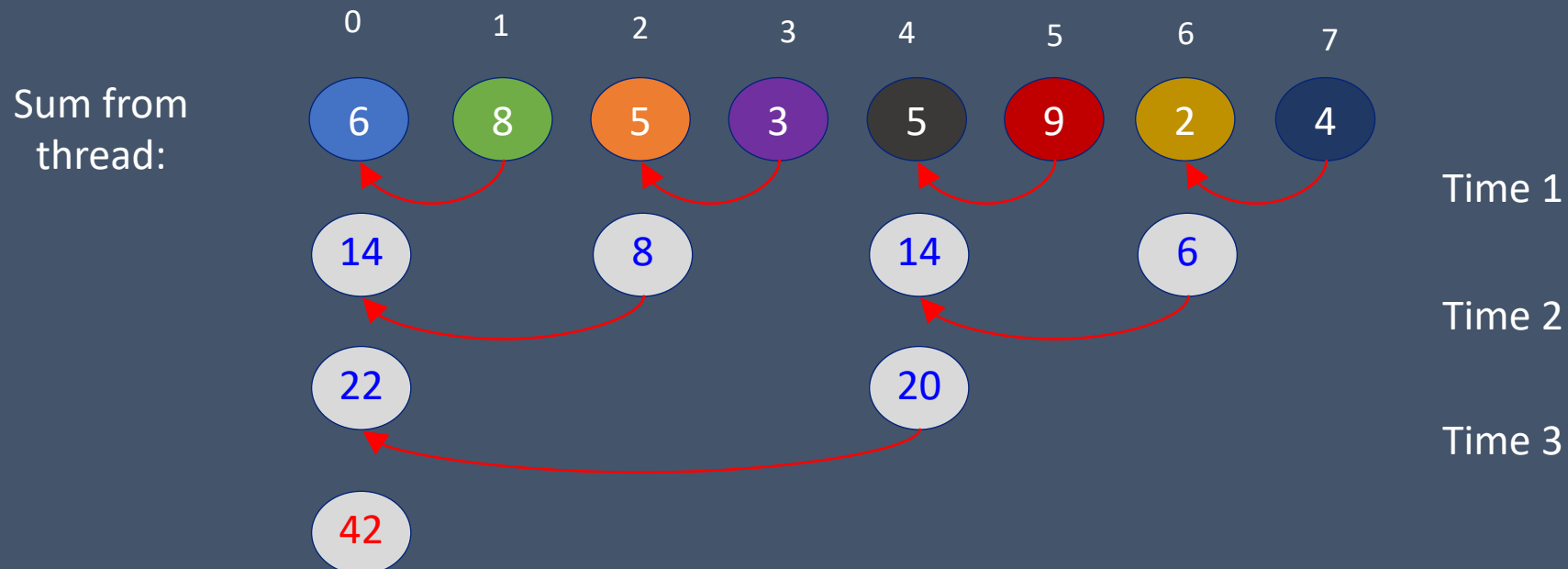
thread₃





Communication Pattern: Reduction

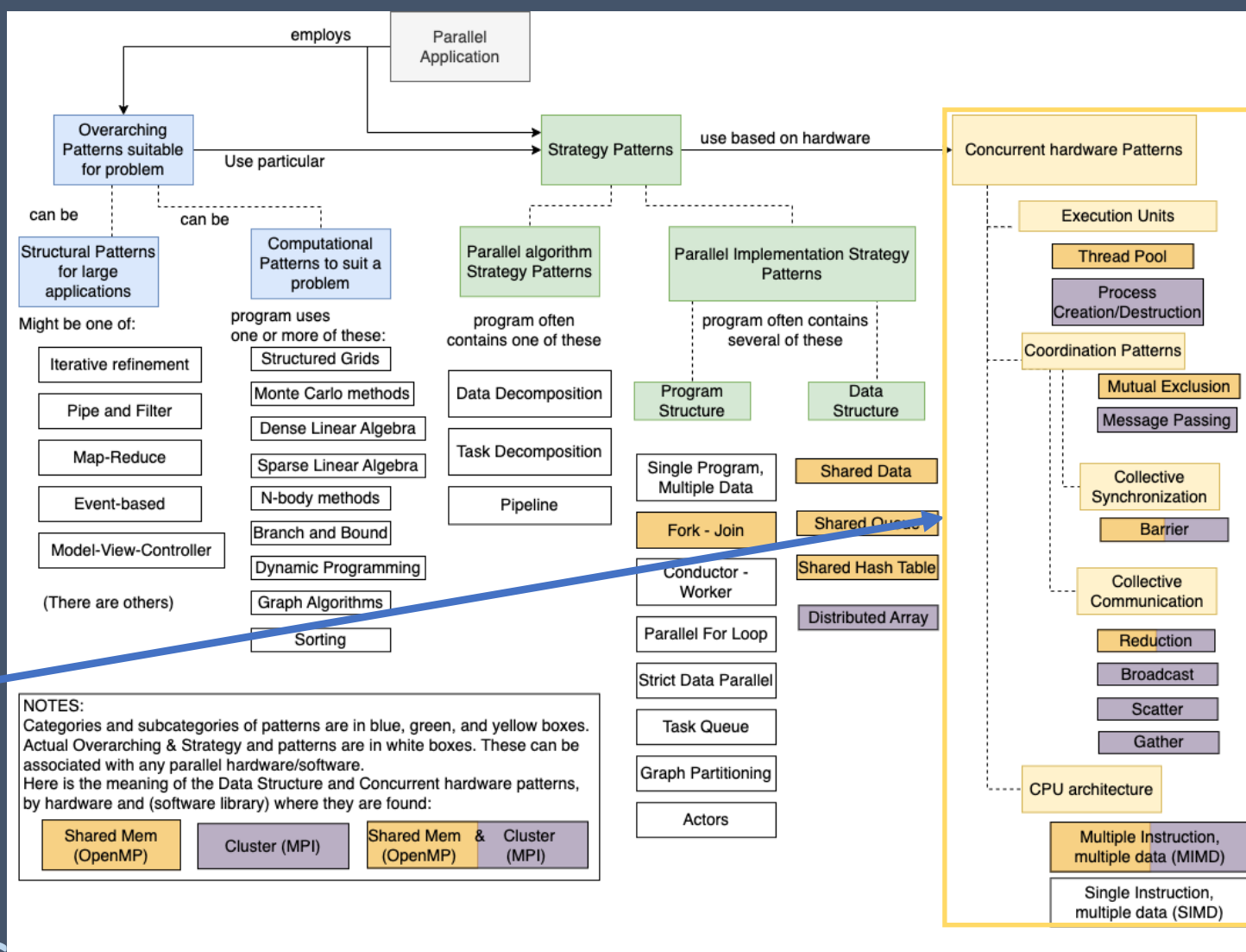
Parallel programs often need to combine the local results of N parallel tasks.
The **reduction** pattern typically does it in $O(\log(N))$ time

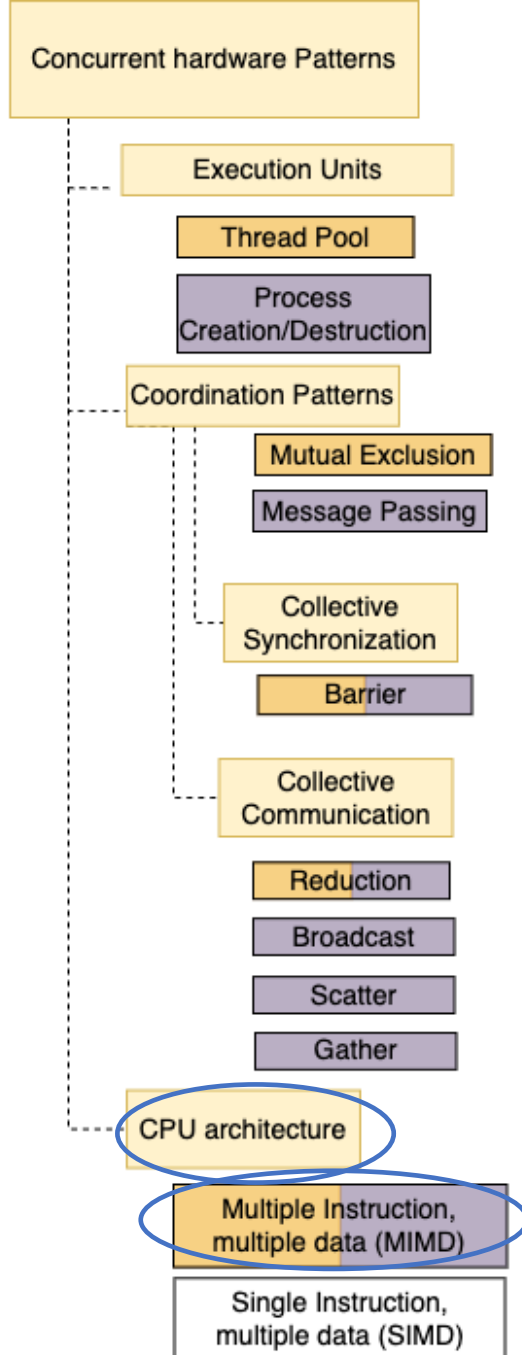


Concurrent Hardware Patterns

Communication and synchronization of tasks

These are usually found inside of the libraries we use to build our code.





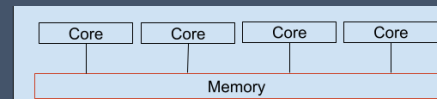
Shared Memory OpenMP & MPI pattern

MIMD:

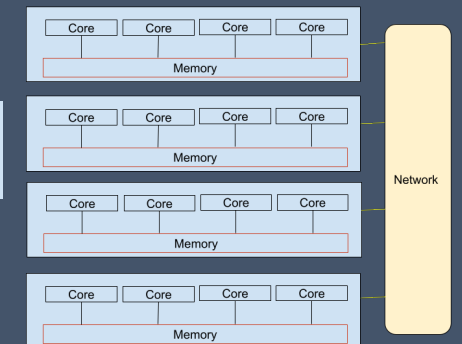
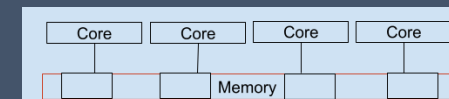
Multiple Instruction, Multiple Data

A consequence of *shared memory CPU architecture*:

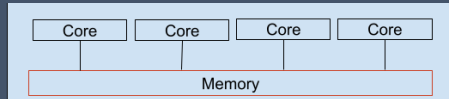
simultaneous instructions accessing shared data



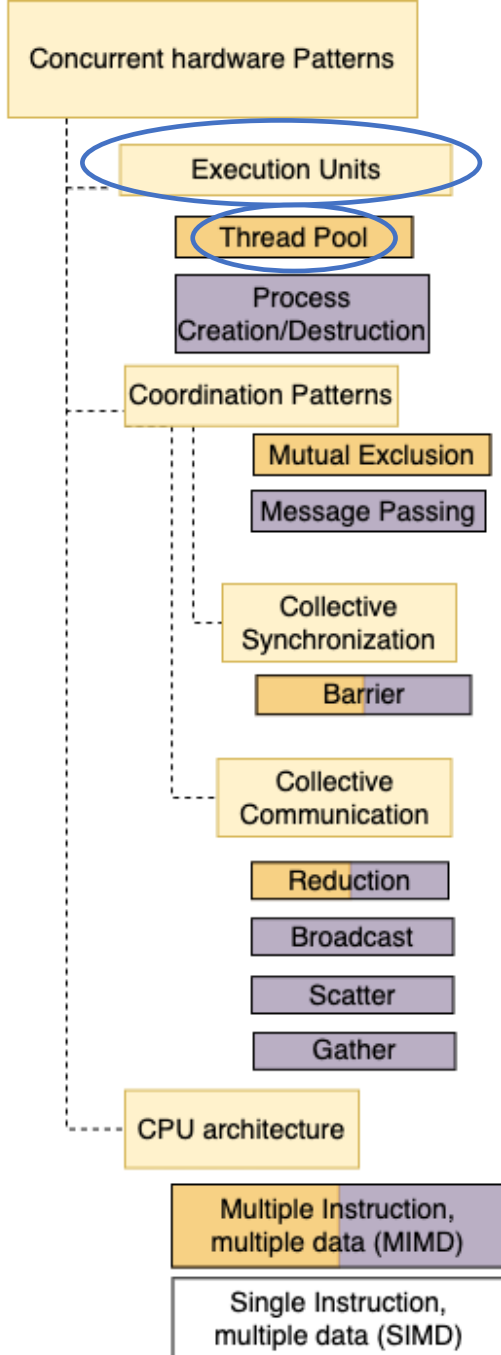
Threads, OpenMP



Processes, MPI



Threads, OpenMP



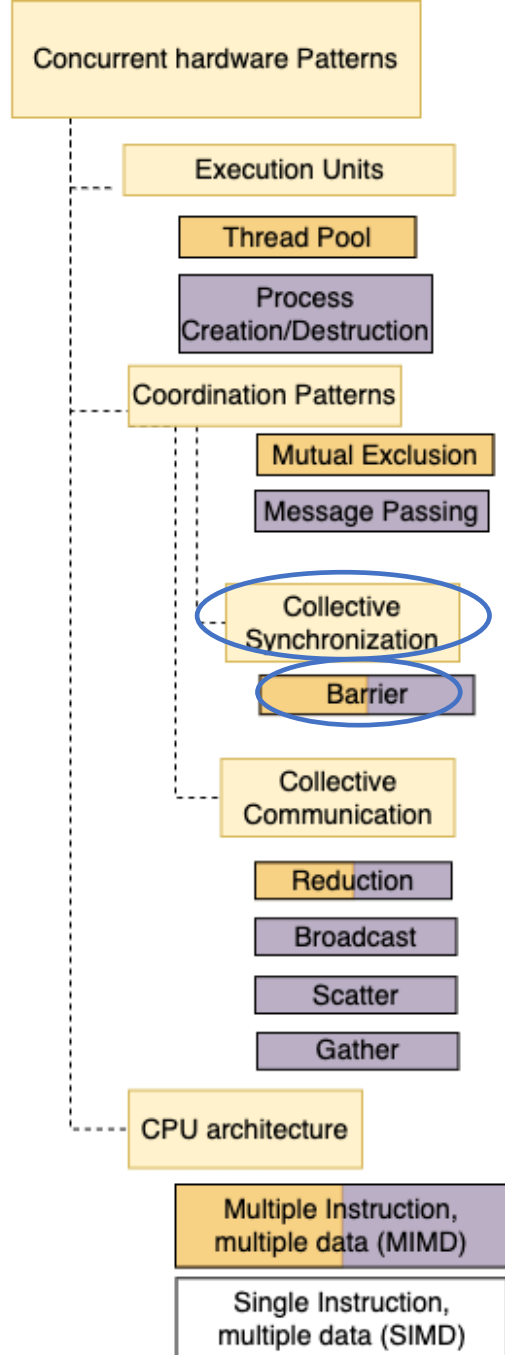
Shared Memory OpenMP patterns

Thread Pool:

Before parallel execution, threads are set up in a 'pool': a group of available threads

In computer science, a **pool** is a collection of resources that are kept ready to use, rather than acquired on use and released afterwards. In this context, *resources* can refer to system resources ...

[https://en.wikipedia.org/wiki/Pool_\(computer_science\)](https://en.wikipedia.org/wiki/Pool_(computer_science))



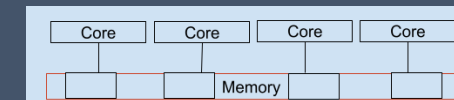
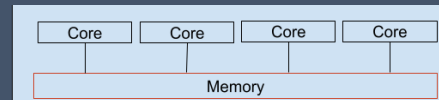
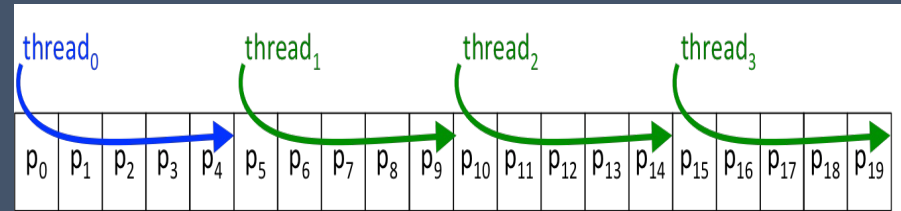
Shared Memory OpenMP & MPI pattern

Barrier:

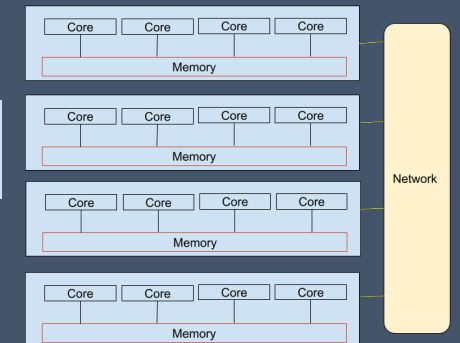
Place in code where all threads or processes must finish before continuing

Code examples in book, activities

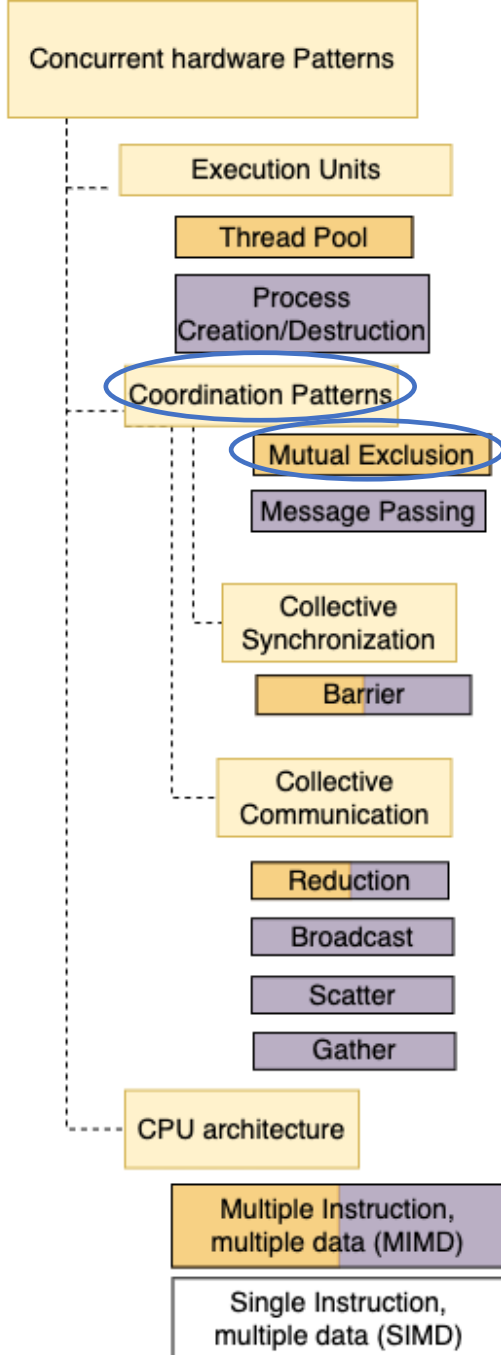
There are implicit barriers also: end of parallel loop, e.g.



Threads, OpenMP



Processes, MPI



Shared Memory OpenMP pattern

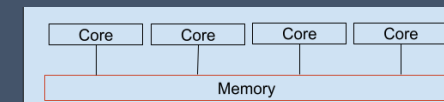
Mutual Exclusion:

Provides a place in code where only one thread can execute at a time

(critical section)

Specific to shared-memory systems where race conditions can occur

Patternlets show two possibilities for this in OpenMP
atomic
critical



Threads, OpenMP

Parallel Patterns in Programs

... are industry-standard techniques and best-practices that have proven useful in many different parallel contexts.

... are built into popular platforms like MPI and OpenMP.

... are likely to be useful for the long-term, regardless of future PDC developments.

... provide a way to organize PDC concepts.

Conclusions

There are many possible starting points in PDC

There are *many* hardware and software options

Getting started is more important than where

Choose a software platform(s) that we can start with:

In first part of course:

C/C++ and shared memory: **OpenMP, OpenACC**

C and “distributed” systems: **MPI**

Patterns offer a much-needed source of stability

You will find them in different types of software on various hardware

Hw 0: Shared Memory with OpenMP

#pragma omp parallel

